

Dispositif de prediction de pannes electriques et de basculement automatique

Rapport Technique – Phase 2 (Version 2.0)

Pipeline de preparation des datasets equilibres pour l'entrainement
ML

Cours	UE INF4258 – Projet (Master 1)
Encadrant	Pr. Paulin Melatagia
Environnement	Hopital ~100 lits, Cameroun
Outils	Python 3 / scikit-learn / imbalanced-learn
Dataset source	dataset_electricity_outage.csv 116 715 lignes – 20 colonnes
Version	2.0 – 17 avril 2026
Statut	4 datasets valides et exportes

Ce que ce rapport apporte par rapport a la version 1.0

La v2.0 corrige cinq lacunes identifiees : (1) colonnes pertinentes manquantes dans les features, (2) absence du timestamp dans les exports test, (3) deux scalers identiques au lieu d'un seul, (4) aucun rapport de verification automatique, (5) justification incomplete du role du preprocessing avant SMOTE. Chaque correction est documentee et numerotee dans ce rapport.

Table des matières

1	Contexte et objectif	2
1.1	Le probleme concret	2
1.2	Structure globale du pipeline	2
2	Dataset source – analyse complete des 20 colonnes	2
2.1	Inventaire et classification de chaque colonne	2
2.2	Desequilibre des classes – chiffres exacts	4
3	Etape 1 – Chargement et tri chronologique	4
4	Etape 2 – Feature engineering temporel (avant le split)	4
4.1	Principe general	4
4.2	La regle causale – le shift(1)	5
4.3	Colonnes concernees par les lags	6
4.4	Pourquoi calculer les features AVANT le split ?	6
5	Etape 3 – Split temporel strict 80/20	7
6	Etape 4 – Selection des features	7
7	Etape 5 – Le preprocessing est-il important avant SMOTE ?	8
7.1	La reponse courte : oui, le preprocessing est obligatoire avant SMOTE	8
7.2	Le probleme sans normalisation	8
7.3	La solution : StandardScaler	9
8	Etape 6 – Application de SMOTE	10
8.1	Fonctionnement de SMOTE	10
8.2	Pourquoi SMOTE est applicable ici malgre les series temporelles . . .	10
8.3	Parametres et justification	10
8.4	Resultats	11
9	Etape 7 – Export des 4 fichiers CSV	11
10	Rapport de verification automatique	12
11	Recapitulatif et tableau des corrections v1 -> v2	13
12	Perspectives – Phase 2	13

1 Contexte et objectif

1.1 Le probleme concret

Un hopital au Cameroun subit des coupures d’électricite imprevuees sur le reseau BTA 220 V / 50 Hz. Des patients en reanimation ou en bloc operatoire sont alimentes par des equipements sous tension continue. Une coupure de 2 secondes peut etre mortelle.

L’objectif n’est pas de detecter la panne quand elle est deja la – c’est **d’agir avant qu’elle survienne**. Cela impose de construire deux programmes (modeles) distincts :

Table 1.1 – Les deux modeles du systeme de basculement

Modele	Cible	Role	Dataset
A	pre_fault	Detecte la chute progressive de tension 6–16 s avant la coupure. Declenche le basculement sur groupe electrogene.	A
B	outage	Confirme que la panne est active. Maintient le basculement jusqu’a retour du courant.	B

1.2 Structure globale du pipeline

Le pipeline se deroule en 7 etapes dans cet ordre strict :

(1) Chargement + tri → (2) Features temporelles → (3) Split 80/20 → (4) Selection features → (5) Normalisation → (6) SMOTE → (7) Export CSV

L’ordre est non-interchangeable : chaque etape depend des precedentes et des violations de cet ordre induisent des fuites d’information (data leakage).

2 Dataset source – analyse complete des 20 colonnes

2.1 Inventaire et classification de chaque colonne

Le dataset contient 20 colonnes. La question « pourquoi telle colonne est incluse ou non dans les features » est fondamentale. Le tableau suivant repond colonne par colonne.

Table 2.1 – Classification de chaque colonne du dataset source

Colonne	Decision	Justification
timestamp	Exclu (ID)	Identifiant temporel. Pas une mesure physique. Conserve uniquement dans les exports test.
cycle	Exclu (ID)	Compteur local reinitialise a chaque session. Identifiant, pas une feature.
cycle_global	Exclu (ID)	Numero de ligne global (= 1 a N). Parfaitement correle a l'ordre temporel – l'inclure reviendrait a donner l'heure au modele sous une autre forme.
voltage_v	Feature	Mesure physique directe (capteur). Signal primaire de panne.
frequency_hz	Feature	Mesure physique directe. La frequence hors tolerance precede souvent une coupure.
power_w	Feature	Mesure physique directe. Tombe a 0 lors d'une coupure.
thd_percent	Feature	Distorsion harmonique. Indicateur de pollution du reseau par les equipements medicaux.
voltage_deviation	Feature	= $voltage_v - 220$. Encode l'ecart a la tension nominale. Redondant avec voltage_v mais enrichit la representation de l'ecart a la norme reglementaire MINEE art. 14.
freq_deviation	Feature	= $frequency_hz - 50$. Meme logique que voltage_deviation .
voltage_out_of_tolerance	Feature	Flag binaire : tension hors $\pm 10\%$. Tres discriminant pour les periodes de coupure et de pre-default.
freq_out_of_tolerance	Feature	Flag binaire : frequence hors $\pm 5\%$. 35,76 % des cycles en journee – signal utile de charge electrique anormale.
hour	Feature	Heure de la mesure (0–23). La probabilite de panne varie selon l'heure (pointe, nuit, etc.).
is_peak_hour	Feature	$= 1$ si $18h \leq h \leq 22h$. Plage de forte demande du reseau.
is_night	Feature	$= 1$ si $22h \leq h \leq 6h$. Regime de

Correction apportee (v2)

Correction v2 – colonnes manquantes en v1 : La version 1 excluait `micro_outage` (correct), mais n'ajoutait des lags que sur 4 colonnes (`voltage_v`, `frequency_hz`, `power_w`, `thd_percent`). La v2 ajoute également des lags sur `voltage_out_of_tolerance` et `freq_out_of_tolerance`, ce qui capture l'historique des periodes d'instabilite binaire – information absente de la v1. Resultat : **56 features** au lieu de 42 en v1.

2.2 Desequilibre des classes – chiffres exacts**Table 2.2** – Distribution des classes dans le dataset brut

Variable	Cas 0	Cas 1	Proportion	Ratio
<code>pre_fault</code>	116 600	115	0,099 %	1 : 1 014
<code>outage</code>	111 441	5 274	4,52 %	1 : 21

3 Etape 1 – Chargement et tri chronologique

```

1 df = pd.read_csv("dataset_electricity_outage.csv",
2                   parse_dates=["timestamp"])
3 df = df.sort_values("timestamp").reset_index(drop=True)

```

Pourquoi ?

Le fichier CSV peut ne pas etre ordonne dans le temps si plusieurs sessions de collecte ont ete concatenees. Le tri par timestamp garantit que toutes les operations suivantes (lags, split) respectent l'ordre chronologique reel.

4 Etape 2 – Feature engineering temporel (avant le split)**4.1 Principe general**

Une mesure isolee a l'instant t ne suffit pas a detecter un precurseur de panne. La tension peut valoir 190 V pour deux raisons tres differentes : une fluctuation normale passagere, ou la 5eme valeur d'une chute progressive depuis 220 V. Seul le contexte temporel permet de distinguer les deux.

On construit donc, pour chaque colonne physique ou indicateur binaire pertinent, deux types de features derivees :

1. **Features de decalage (lags)** : $x_{t-1}, x_{t-2}, \dots, x_{t-5}$ – les 5 valeurs precedentes (10 secondes d'historique).

2. **Statistiques glissantes** : moyenne μ_t et ecart-type σ_t sur les 10 cycles precedents (20 secondes).

4.2 La regle causale – le `shift(1)`

Regle critique

La regle causale fondamentale est que la feature a l'instant t ne doit contenir aucune information sur la valeur a l'instant t . En code :

```
1  # CORRECT : shift(1) exclut la valeur courante
2  df[f"{col}_roll_mean"] = df[col].shift(1).rolling(WINDOW).
   mean()
3
4  # FAUX : sans shift(1), la valeur t est incluse dans sa
   propre feature
5  df[f"{col}_roll_mean"] = df[col].rolling(WINDOW).mean()  #
   fuite !
```

Sans `shift(1)`, la moyenne glissante a l'instant t inclut x_t elle-meme. Cela revient a donner au modele la reponse pendant l'examen – les performances en test seraient artificiellement elevees mais le modele echouerait en production.

La formule mathematique correcte pour la moyenne glissante est :

$$\mu_t = \frac{1}{W} \sum_{i=1}^W x_{t-i}, \quad W = 10 \quad (1)$$

4.3 Colonnes concernees par les lags

Table 4.1 – Colonnes recevant des features temporelles

Colonne	Lags (5)	Rolling (2)	Justification
voltage_v	✓	✓	Signal primaire – la chute est le signe principal de <code>pre_fault</code>
frequency_hz	✓	✓	Instabilite frequence precede les pannes
power_w	✓	✓	Tombe a 0 lors d'une coupure
thd_percent	✓	✓	Montee du THD annonce surcharge des equipements
voltage_out_of_tolerance	✓	✓	Combien de cycles recents etaient hors tolerance ?
freq_out_of_tolerance	✓	✓	Idem pour la frequence
<i>Total nouvelles colonnes</i>			$6 \times (5 + 2) = \mathbf{42}$ colonnes

4.4 Pourquoi calculer les features AVANT le split ?

Pourquoi ?

Si on calculait les features apres le split, les 10 premieres lignes de l'ensemble de test n'auraient pas d'historique : leur passe (les 10 dernieres lignes du train) serait dans un autre dataframe. On serait force de supprimer ces 10 lignes ou d'inventer un historique.

En calculant les features sur le dataset complet avant le split, chaque ligne – meme la premiere du test – dispose de son vrai historique temporel. Seul le calcul de la normalisation (`StandardScaler`) se fait apres le split (voir etape 5).

Resultat

Apres suppression des 10 premieres lignes sans historique suffisant : **116 705 lignes conservees, 62 colonnes** au total (20 originales + 42 derivees).

5 Etape 3 – Split temporel strict 80/20

```

1 split = int(len(df_feat) * 0.80)
2 train = df_feat.iloc[:split].copy()    # 93 364 lignes
3 test  = df_feat.iloc[split:].copy()    # 23 341 lignes

```

Regle critique

Jamais de `shuffle=True`, jamais de `train_test_split` de sklearn avec l'option par défaut. Ces fonctions melangent aleatoirement les indices, brisant la chronologie. Un modele entraine sur des donnees du 12 avril et evalue sur celles du 9 avril a deja vu le futur – ses performances seront irrealistes.

Table 5.1 – Resultats du split temporel

Parametre	Train	Test
Lignes	93 364	23 341
Periode	07/04 → 11/04 14h01	11/04 14h01 → 12/04 21h14
Proportion	80 %	20 %

6 Etape 4 – Selection des features

Apres le split, on definit la liste des 56 colonnes qui seront les entrees des deux modeles. Les colonnes exclues sont les identifiants (`timestamp`, `cycle`, `cycle_global`) et les trois cibles (`outage`, `micro_outage`, `pre_fault`).

```

1 COLS_EXCLUDES = ["timestamp", "cycle", "cycle_global",
2                  "outage", "micro_outage", "pre_fault"]
3 FEATURES = [c for c in df_feat.columns if c not in COLS_EXCLUDES]
4 # len(FEATURES) = 56

```


Table 6.1 – Repartition des 56 features par categorie

Categorie	Nb	Exemples
Mesures physiques	4	voltage_v, frequency_hz, power_w, thd_percent
Ecart a la norme	2	voltage_deviation, freq_deviation
Flags de tolerance	2	voltage_out_of_tolerance, freq_out_of_tolerance
Contexte temporel	3	hour, is_peak_hour, is_night
Memoire du reseau	3	outage_count, micro_outage_count, cycles_since_last_outage
Lags et statistiques glissantes	42	voltage_v_lag1, voltage_v_roll_mean, ...
Total	56	

7 Etape 5 – Le preprocessing est-il important avant SMOTE ?

Cette question merite une reponse detaillee parce qu'elle est souvent mal comprise.

7.1 La reponse courte : oui, le preprocessing est obligatoire avant SMOTE

SMOTE genere des exemples synthetiques en interpolant entre les k plus proches voisins de chaque exemple positif. La notion de « voisin » est mesuree par la **distance euclidienne** :

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{f=1}^{56} (x_{i,f} - x_{j,f})^2} \quad (2)$$

7.2 Le probleme sans normalisation

Les 56 features ont des ordres de grandeur radicalement differents :

Table 7.1 – Ordres de grandeur des features sans normalisation

Feature	Min	Max	Amplitude
power_w	0	200 000	200 000
voltage_v	0	260	260
frequency_hz	0	66	66
thd_percent	0	20	20
is_night	0	1	1

Dans la formule de distance euclidienne, un ecart de 100 W sur `power_w` pese autant qu'un ecart de 100 sur `voltage_v`, `frequency_hz`, `thd_percent` et `is_night` reunis. En pratique, la distance est dominee a plus de 99 % par `power_w` et ses lags.

Conséquence concrète : SMOTE choisit des voisins qui sont proches en puissance mais eventuellement tres differents en tension ou en frequence. Les exemples synthetiques generes auront une puissance plausible mais une tension ou une frequence incoherente. Le modele apprend un signal bruite et ses performances se degradent.

7.3 La solution : StandardScaler

Le `StandardScaler` transforme chaque feature x_f en :

$$x'_f = \frac{x_f - \mu_f}{\sigma_f} \quad (3)$$

ou μ_f est la moyenne et σ_f l'ecart-type de la feature f calcules sur le train. Apres transformation, toutes les features ont une moyenne de 0 et un ecart-type de 1. Chaque feature contribue equitalement au calcul de distance.

```

1 scaler = StandardScaler()
2
3 # fit() : calcule moyenne et ecart-type sur le TRAIN uniquement
4 # transform() : applique la meme normalisation
5 X_train_scaled = scaler.fit_transform(train[FEATURES]) # fit +
   transform
6 X_test_scaled = scaler.transform(test[FEATURES]) #
   transform UNIQUEMENT

```

Regle critique

Regle absolue : fit() sur le train uniquement.

Si on appelait `scaler.fit(df_feat[FEATURES])` sur l'ensemble complet (train + test), les parametres μ_f et σ_f seraient calcules en incluant les donnees du test. Le test « informerait » le train sur sa propre distribution – c'est une fuite d'information. La bonne pratique est : `fit_transform` sur le train, `transform` (sans fit) sur le test.

Correction apportee (v2)

Correction v2 – un seul scaler : La v1 instanciat `scaler_A` et `scaler_B` separement, appliques sur les memes features du meme train. Les deux scalers produisaient des parametres identiques (calcules sur les memes donnees). C'etait redondant et trompeur. La v2 utilise un seul `scaler` commun, plus lisible et semantiquement correct.

Resultat

Verification apres normalisation :

Moyenne des features du train : $-0,000000$ (attendu : ≈ 0)

Ecart-type des features du train : $1,000000$ (attendu : ≈ 1)

8 Etape 6 – Application de SMOTE

8.1 Fonctionnement de SMOTE

Pour chaque exemple positif $\mathbf{x}_i$ dans l'ensemble d'entrainement, SMOTE :

1. Identifie ses k plus proches voisins positifs.
2. Choisit aleatoirement l'un d'eux, $\mathbf{x}_{n_j}$.
3. Genere un exemple synthetique :

$$\mathbf{x}_{\text{synth}} = \mathbf{x}_i + \lambda \cdot (\mathbf{x}_{n_j} - \mathbf{x}_i), \quad \lambda \sim \mathcal{U}(0, 1) \quad (4)$$

Le resultat est un exemple qui se situe **entre** deux exemples reels dans l'espace des features – jamais en dehors.

8.2 Pourquoi SMOTE est applicable ici malgre les series temporelles

SMOTE n'est pas concu pour les series temporelles brutes. Mais dans ce pipeline, SMOTE n'opere **pas sur des sequences** : il opere sur des **vecteurs de 56 features**. Chaque vecteur encode deja son propre contexte historique via les lags. SMOTE interpole entre des vecteurs dans un espace de 56 dimensions – la coherence temporelle est integree dans chaque vecteur.

8.3 Parametres et justification

Table 8.1 – Parametres SMOTE pour les deux modeles

Parametre	Modele A	Modele B	Justification
sampling_strategy	0,10	0,33	1 :10 pour A, 1 :3 pour B
k_neighbors	3	5	k=3 car 92 positifs seulement
random_state	42	42	Reproductibilite

Pourquoi k=3 pour le Modele A ? Avec seulement 92 positifs reels et k=5, certains positifs pourraient ne pas avoir 5 voisins positifs accessibles – SMOTE leverebbe une erreur. k=3 est le minimum sur pour ce volume de positifs.

Pourquoi ne pas viser un ratio 1 :1 ? Creer 93 000 exemples synthetiques a partir de 92 reels revient a inventer 99,9 % de la classe positive. Le modele n'apprendrait plus la physique du precurseur de panne mais une extrapolation mathematique eloignee de la realite. Un ratio 1 :10 est un compromis entre representation suffisante et fidelite physique.

8.4 Resultats

Table 8.2 – Distribution des classes avant et apres SMOTE

Dataset	Avant		Apres		Synthetiques generes
	Pos.	Neg.	Pos.	Neg.	
A (pre_fault)	92	93 272	9 327	93 272	9 235
B (outage)	4 491	88 873	29 328	88 873	24 837

9 Etape 7 – Export des 4 fichiers CSV

```

1 # Train A : features normalisees + SMOTE + colonne target
2 df_train_A = pd.DataFrame(X_res_A, columns=FEATURES)
3 df_train_A["target"] = y_res_A
4 df_train_A.to_csv("train_pre_fault.csv", index=False)
5
6 # Test A : features normalisees + timestamp (diagnostic) +
  target
7 df_test_A = pd.DataFrame(X_test_scaled, columns=FEATURES)
8 df_test_A.insert(0, "timestamp", test["timestamp"].values)
9 df_test_A["target"] = y_test_A
10 df_test_A.to_csv("test_pre_fault.csv", index=False)

```

Correction apportee (v2)

Correction v2 – timestamp conserve dans les exports test : La v1 exportait les tests sans le `timestamp`. En Phase 2, lors de l'analyse des erreurs du modele, il est essentiel de savoir *quand* les fausses alarmes et les pannes manquees surviennent (nuit ? pointe ?). La v2 insere le `timestamp` en premiere colonne des fichiers test uniquement – les fichiers train n'en ont pas besoin (SMOTE a de toute facon brise l'ordre temporel).

Table 9.1 – Description des 4 fichiers CSV produits

Fichier	Lignes	Cols	Pos.	Neg.
train_pre_fault.csv	102 599	57	9 327	93 272
test_pre_fault.csv	23 341	58	23	23 318
train_outage.csv	118 201	57	29 328	88 873
test_outage.csv	23 341	58	783	22 558

Train : 57 colonnes (56 features + target). Test : 58 colonnes (timestamp + 56 features + target).

10 Rapport de verification automatique

Correction apportee (v2)

Correction v2 – verification automatique en fin d’execution : La v1 n’effectuait aucun controle de coherence. La v2 ajoute des assertions qui levent une erreur si une invariante est violee.

Trois proprietes invariantes sont verifiees par assertion :

1. **Test non modifie par SMOTE :** le nombre de lignes des fichiers test doit etre egal au nombre de lignes de l’ensemble test brut (23 341).
2. **SMOTE n’a sur-echantillonne que les positifs :** le nombre de negatifs dans le train est identique avant et apres SMOTE (93 272 pour A, 88 873 pour B).
3. **Aucune valeur manquante :** les 4 fichiers CSV sont complets.

Tous les tests sont passes avec succes lors de l’execution.

11 Recapitulatif et tableau des corrections v1 -> v2

Table 11.1 – Recapitulatif des corrections apportees en version 2

#	Probleme en v1	Impact	Correction v2
C1	Lags seulement sur 4 colonnes	Information temporelle des flags de tolerance ignoree	Lags + rolling sur 6 colonnes (42 nouvelles features)
C2	Timestamp absent des exports test	Impossible d'analyser les erreurs dans le temps	Timestamp en colonne 1 des fichiers test
C3	Deux scalers identiques (scaler_A, scaler_B)	Redondance, confusion semantique	Un seul scaler commun
C4	Aucun rapport de verification	Fuites possibles non detectees	3 assertions automatiques en fin d'execution
C5	Role du preprocessing non justifie	Incomprehension possible du besoin de normaliser	Section 7 detaillee avec calcul d'impact

Table 11.2 – Comparaison des dimensions v1 vs v2

Metrique	Version 1	Version 2
Features par ligne	42	56
Lignes train A (post-SMOTE)	102 597	102 599
Lignes train B (post-SMOTE)	118 198	118 201
Colonnes fichiers test	43	58 (avec timestamp)
Scalers instances	2	1
Assertions de verification	0	3

12 Perspectives – Phase 2

Les 4 datasets sont prêts pour l'entraînement. En Phase 2 :

1. **Modeles** : Random Forest ou Gradient Boosting sur chaque dataset.

2. **Seuil de decision** : ne pas utiliser le seuil par défaut de 0,5. Pour un hopital, manquer une panne est plus grave que declencher une fausse alarme. Le seuil devrait etre abaisse a 0,2–0,3 pour le Modele A.
3. **Metriques** : Recall, Precision, F1-score et PR-AUC. Jamais l’accuracy sur des donnees desequilibrees.
4. **Analyse temporelle des erreurs** : utiliser le `timestamp` des fichiers test pour verifier si les faux negatifs surviennent preferentiellement la nuit ou en pointe.

Références

- [1] Chawla N.V., Bowyer K.W., Hall L.O., Kegelmeyer W.P., *SMOTE : Synthetic Minority Over-sampling Technique*, JAIR, vol. 16, pp. 321–357, 2002.
- [2] Moukengue Imano A., *Problemes d’electrification urbaine au Cameroun*, Laboratoire LEEAT, Universite de Douala, 2015.
- [3] Lemaitre G., Nogueira F., Aridas C.K., *Imbalanced-learn : A Python Toolbox*, JMLR, vol. 18, 2017.
- [4] Ministere de l’Energie et de l’Eau du Cameroun, *Arrete n° 00000013/MINEE*, Yaounde, 2009.